



Certified Kubernetes Administrator (CKA): Installing Clusters with kubeadm

Preparing Linux hosts and container runtime dependencies



Tim Warner

Principal Author | IT Ops | Pluralsight

TechTrainerTim.com



Four framing questions



Why does kubeadm require specific kernel modules and sysctl parameters?



What is containerd's role as the CRI-compliant container runtime?



How do I install and pin kubeadm, kubelet, and kubectl at a specific version?



How do I verify all prerequisites across multiple nodes before bootstrapping?





“Infrastructure provisioned three bare Ubuntu VMs for your production cluster. You have root access and a deadline: control plane up by end of day. Before you touch kubeadm, these hosts need specific kernel modules, network settings, and a container runtime. Skip any step and kubeadm init fails with cryptic errors.”



**Why does kubeadm require specific
kernel modules and sysctl parameters?**



Kernel modules and sysctl: the networking foundation

overlay:

- Enables overlay filesystem support required by container runtimes (containerd) for image layers

br_netfilter:

- Enables iptables to inspect bridged network traffic

net.ipv4.ip_forward=1:

- Enables packet forwarding between network interfaces (required for pod-to-pod communication across nodes)

net.bridge.bridge-nf-call-iptables=1; net.bridge.bridge-nf-call-ip6tables=1:

- Enables bridged traffic filtering

Persistence:

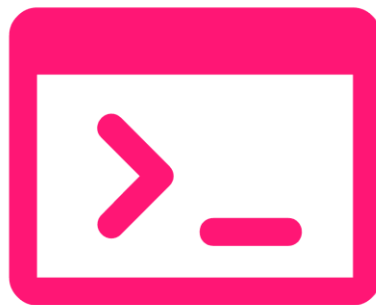
- Persist modules in /etc/modules-load.d/ and sysctl in /etc/sysctl.d/

Warning:

- Without these, kubeadm preflight checks fail or cluster networking breaks later



kubeadm prerequisites: what must be correct before init



Swap must be disabled

```
sudo swapoff -a
```

All nodes must run identical kubeadm, kubelet, kubectl versions

Kernel modules + sysctl must be configured (networking foundation)

kubeadm does NOT install containerd or CNI — you must install both



Configure kernel modules and sysctl for Kubernetes networking

```
# Load required kernel modules (runtime)
sudo modprobe overlay          # Required for container image layers (containerd)
sudo modprobe br_netfilter     # Allows iptables to see bridged traffic

# Persist modules across reboots
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

# Configure sysctl networking parameters
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.ipv4.ip_forward = 1          # Enable packet forwarding between interfaces
net.bridge.bridge-nf-call-iptables = 1 # Ensure bridged traffic hits iptables
# net.bridge.bridge-nf-call-ip6tables = 1 # Optional (IPv6 environments)
EOF

# Apply sysctl settings immediately
sudo sysctl --system

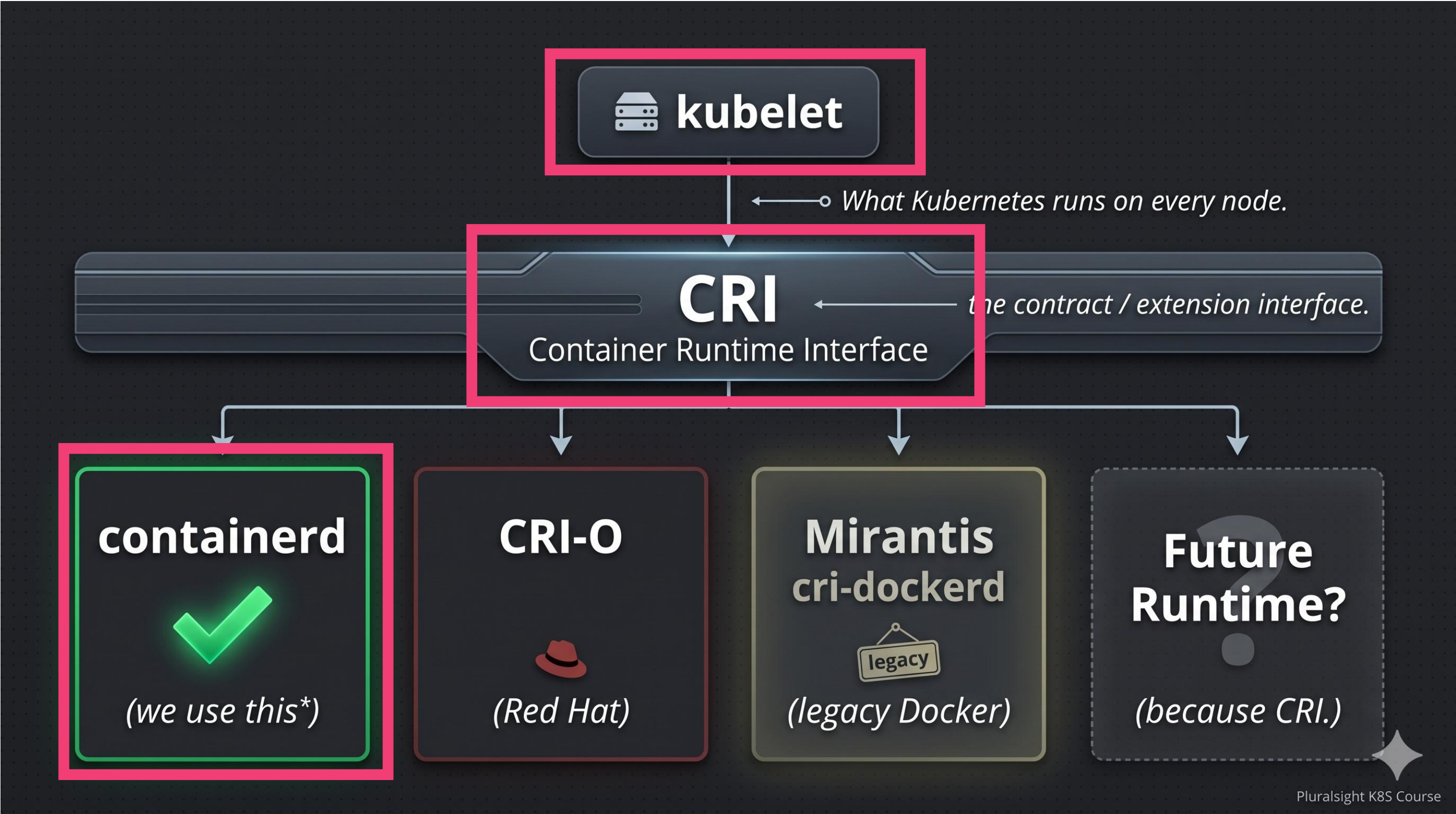
# Quick verification
lsmod | grep -E 'overlay|br_netfilter' # Modules loaded
sysctl net.ipv4.ip_forward              # Should return = 1
```



What is containerd's role as the CRI-compliant container runtime?



containerd and the Container Runtime Interface (CRI)



containerd and the Container Runtime Interface (CRI)

CRI is the abstraction (the contract). containerd is one implementation -- CRI-O is another.

containerd replaced Docker as the default runtime (dockershim removed in 1.24)

Install containerd, generate default config with containerd config default

Ensure containerd and kubelet use the same cgroup driver (systemd on modern Linux)

Verify with `systemctl status containerd` and `crictl info`



Install and configure containerd for kubeadm (CRI + cgroups)

```
# Install containerd (Ubuntu example)
```

```
sudo apt-get update
```

```
sudo apt-get install -y containerd
```

```
# Generate default configuration
```

```
sudo mkdir -p /etc/containerd
```

```
containerd config default | sudo tee /etc/containerd/config.toml
```

```
# Align cgroup driver -- #1 CAUSE OF kubeadm init FAILURES
```

```
# SystemdCgroup = true MUST match kubelet's driver (systemd on modern Linux)
```

```
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' \  
/etc/containerd/config.toml
```

```
# Restart containerd to apply config
```

```
sudo systemctl restart containerd
```

```
sudo systemctl enable containerd
```

```
# Verify container runtime + CRI endpoint
```

```
systemctl status containerd      # Service should be active (running)
```

```
sudo crictl info                  # Confirms CRI is working and shows runtime details
```



Configure crictl before you need it (exam-day lifesaver)

```
# Tell crictl which runtime socket to talk to
# Without this, crictl guesses -- and often guesses wrong on exam VMs
sudo crictl config --set runtime-endpoint=unix:///run/containerd/containerd.sock

# Verify the config was written
cat /etc/crictl.yaml

# Expected output:
# runtime-endpoint: unix:///run/containerd/containerd.sock

# Now crictl runs cleanly -- no warnings, no endpoint guessing
sudo crictl info      # JSON dump of runtime details
sudo crictl ps        # Running containers (post-bootstrap)
sudo crictl images    # Cached images on this node
```



**How do I install and pin kubeadm,
kubelet, and kubectl at a specific
version?**



Installing and pinning the Kubernetes packages



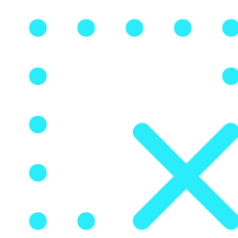
Add the official
Kubernetes apt repository
with signed GPG key



Install kubeadm, kubelet,
and kubectl at v1.35



apt-mark hold kubeadm
kubelet kubectl on ALL
THREE NODES (CP + both
workers) immediately after
install



kubelet starts but crashes
until kubeadm init
configures it



Identical versions on every
node — verify with `dpkg -l
kubeadm kubelet kubectl`



How do I verify all prerequisites across multiple nodes before bootstrapping?



Verifying prerequisites across all cluster nodes

Every node needs identical kernel modules, sysctl, containerd, and packages

Run a verification script that checks every prerequisite on each node

Check swap is disabled: swapon --show should return nothing

Check containerd: systemctl status containerd and crictl info

Check packages: dpkg -l kubeadm kubelet kubectl shows pinned versions



Demo

Prepare three Linux VMs for kubeadm from bare Ubuntu hosts





“All three nodes passed the prerequisite check. Kernel modules loaded, sysctl configured, containerd running with the right cgroup driver, and packages pinned. Now I am confident these hosts will not surprise us during kubeadm init.”





From Globomantics to you



Kernel modules are non-negotiable

overlay and br_netfilter must be loaded AND persisted before kubeadm init



Systemd Cgroup alignment prevents silent failures

Mismatched cgroup drivers between containerd and kubelet cause pods to fail without clear errors



Version pinning protects cluster stability

apt-mark hold prevents the OS package manager from upgrading Kubernetes behind your back



Verify every node before bootstrapping

Run a prerequisite check script on all nodes -- one misconfigured node derails the entire cluster

